

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Operating System (BCSE303P)

Course Project – Review 1

L15+L16

Faculty: Dr. Anisha M. Lal

Adaptive CPU Scheduling Using Predicted Burst Lengths

Submitted by

Shivaansh Aggarwal	24BCE0883
Devansh Sharma	24BCE0717

Title and Team

Project Title	Adaptive CPU Scheduling Using Predicted Burst Lengths
Course	Operating System (BCSE303P), Slot L15+L16
Faculty	Dr. Anisha M. Lal
Team Members	Shivaansh Aggarwal (24BCE0883) Devansh Sharma (24BCE0717)

Abstract

The CPU scheduler decides which ready process gets the processor next and for how long it may keep it. The algorithms covered in the course, First-Come-First-Served (FCFS), Shortest-Job-First (SJF) and Round Robin (RR), are all static: each is configured once and then applied the same way no matter how the workload behaves. SJF gives the lowest average waiting time but needs the length of the next CPU burst, which is not known at the time of the decision. Operating systems therefore estimate it with exponential averaging, an estimate that lags badly whenever a process changes its behaviour. Round Robin, on the other hand, uses a time quantum that is chosen by hand and never revisited.

This project builds a CPU scheduler that predicts the next CPU burst of every process using a small machine learning model trained on the process's own history, and then uses that prediction in two places: to decide which process runs next, with an ageing term so that long processes do not starve, and to resize the time quantum continuously while the system runs. The scheduler is implemented in Python inside a discrete-event simulator and compared against FCFS, SJF, SRTF, Round Robin with several quanta, a multilevel feedback queue, and an oracle that knows every burst in advance. This document covers the background, the literature that was studied, and the gaps in existing work that the project sets out to address.

1. Introduction

1.1 Background

In a multiprogrammed system several processes are ready to run at the same time, but a single CPU can execute only one of them. The short-term scheduler picks one of the ready processes, and this decision is made thousands of times every second. Its quality shows up directly in the numbers a user cares about: how long processes wait in the ready queue, how long it takes for a process to finish (turnaround time), how quickly an interactive program first responds, and how much CPU time is lost to context switching [1].

Every process alternates between CPU bursts, where it computes, and I/O bursts, where it waits for a disk, network or user. Some processes are CPU-bound and have long bursts; others are I/O-bound or interactive and have many short ones. A good scheduler has to deal with all of these at once.

1.2 Motivation

The standard algorithms each handle this mix in a fixed way:

- **FCFS** runs processes in arrival order. It ignores burst length completely, so a single long process at the head of the queue makes every short process behind it wait (the convoy effect). With bursts of 24, 3 and 3 ms arriving together, the average waiting time is 17 ms.
- **SJF** runs the shortest burst first and is provably optimal for average waiting time. For the same three processes it gives an average of only 3 ms. The difficulty is that SJF needs the length of the next CPU burst, and that value does not exist yet when the scheduler has to decide.
- **Round Robin** gives each process a fixed time slice, the quantum. A small quantum makes the system responsive but causes a large number of context switches; a large quantum reduces switching but makes RR behave like FCFS. The quantum is set once, when the system is configured.

To approximate SJF, operating systems estimate the next burst with exponential averaging [1]:

$$\tau_{n+1} = \alpha\, t_n + (1 - \alpha)\, \tau_n$$

where t_n is the length of the most recent burst and τ_n the previous estimate. This works while a process behaves consistently, but it reacts slowly when a process changes phase, for example a program that first reads a large file (short bursts) and then processes it (long bursts). For the burst sequence 2, 2, 2, 2, 60, 60, 60 with $\alpha = 0.5$, the estimates are 2, 31, 45.5 and 52.75, so the process is scheduled on a wrong estimate for three to four bursts in a row. The estimate also uses only one piece of information, the process's own past burst lengths, even though the system knows much more about the process, such as how long it blocks for I/O and how much its bursts vary.

While reading about these algorithms, one thing stood out: the textbook already gives a rule for choosing the quantum, namely that about 80% of CPU bursts should be shorter than the time quantum [1]. But applying this rule needs the distribution of burst lengths, which no scheduler measures, so in practice a person picks a value once. This led us to the idea of the project: if the scheduler could predict upcoming bursts reasonably well, the same prediction could be used both to order the ready queue and to apply the 80% rule automatically, all the time.

1.3 Problem Statement

Classical CPU scheduling algorithms rely either on information that is not available (the next burst length, in SJF) or on parameters that are fixed in advance (the quantum, in Round Robin). The standard burst estimate, exponential averaging, uses a single feature and lags after a change in process behaviour. The problem addressed by this project is to design a scheduler that predicts the next CPU burst of each process from its observed history, uses the prediction to order the ready queue without starving long processes, and adjusts the time quantum while the

system runs, and then to measure honestly whether this performs better than the classical algorithms under a range of workloads.

1.4 Objectives

1. Build a simulation of a single-CPU system that runs many processes and can run any scheduling algorithm on exactly the same workload.
2. Implement FCFS, SJF, SRTF, Round Robin and a multilevel feedback queue, and verify the simulator against hand-computed textbook results.
3. Design and train a small model that predicts a process's next CPU burst from its past behaviour, and compare it against exponential averaging.
4. Use the prediction to decide which process runs next and to adjust the time quantum while the system runs.
5. Compare the proposed scheduler with the classical algorithms across different types of workload, and report where it helps and where it does not.

1.5 Scope

The project covers a single-CPU discrete-event simulator written in Python, synthetic workloads with CPU-bound, I/O-bound, interactive and phase-changing processes, the classical baseline algorithms, the burst prediction model, and the proposed adaptive scheduler. Changes to a real operating system kernel, multi-core scheduling, real-time deadlines and memory or cache effects are outside the scope, so that the work stays focused and can be evaluated carefully within the course timeline.

2. Literature Survey

The survey covers four groups of work: the classical algorithms and their well-known refinements, Round Robin variants that change the time quantum dynamically, methods that predict CPU burst lengths, and more recent learning-based approaches to scheduling.

2.1 Classical scheduling and its refinements

Silberschatz, Galvin and Gagne [1] describe FCFS, SJF, SRTF, priority scheduling and Round Robin, along with exponential averaging for estimating the next burst and the rule of thumb that 80% of bursts should fit within one quantum. Arpaci-Dusseau and Arpaci-Dusseau [2] explain the multilevel feedback queue (MLFQ), which approximates SJF without any burst estimate: a process that uses up its whole time slice moves to a lower-priority queue, and all processes are periodically moved back to the top queue so that none starve. The idea of multiple priority levels goes back to the Compatible Time-Sharing System described by Corbató, Merwin-Daggett and Daley [3]. Waldspurger and Weihl [4] proposed lottery scheduling, a proportional-share approach in which each process holds tickets and the winner of a random draw runs next.

These methods are robust and widely used, but none of them tries to learn how long a process will run.

2.2 Dynamic time quantum in Round Robin

A large number of papers try to fix the fixed-quantum problem of Round Robin. Matarneh [5] proposed Self-Adjustment Round Robin (SARR), where the quantum in each cycle is set to the median of the burst times of the processes currently in the queue. Noon, Kalakech and Kadry [6] proposed a similar algorithm that sets the quantum to the mean of the burst times of the waiting processes, and showed through MATLAB simulations that it lowers average waiting and turnaround time compared with standard Round Robin. Mohanty et al. [7] proposed Priority Based Dynamic Round Robin (PBDRR), which computes an “intelligent” time slice for each process from its priority and burst time and changes it after every round, reducing context switches, waiting time and turnaround time for soft real-time systems. Helmy and Dekdouk [8] proposed Burst Round Robin, which keeps the low overhead of Round Robin while giving shorter jobs priority, and adds a weight adjustment for processes that lose CPU time while blocked on I/O.

All of these show that adapting the quantum helps. However, they compute the quantum from the burst times of the processes in the queue, which are treated as known values, and they are usually evaluated on small sets of CPU-only processes.

2.3 Predicting CPU burst length

Negi and Kumar [9] applied machine learning to Linux process scheduling. Using the Weka toolkit, they learned how much of its time slice a known program uses, found that the C4.5 decision tree worked best, and modified the Linux 2.4 kernel scheduler to give processes customised time slices. They estimated a reduction of about 1.4% to 5.8% in turnaround time, mainly from fewer context switches. Pourali and Rahmani [10] used a fuzzy inference system to predict the next CPU burst for Shortest-Process-Next scheduling, as a replacement for the exponential average. In a related area, Smith, Foster and Taylor [11] predicted the run times of parallel jobs from the history of similar past jobs and showed that such predictions are more accurate than user estimates.

2.4 Learning-based scheduling

More recent work uses heavier learning methods. Mao et al. [12] built DeepRM, which uses deep reinforcement learning to learn how to pack jobs with several resource demands in a cluster, and found that it performs comparably to standard heuristics. Zhang et al. [13] survey the use of machine learning and large language models inside operating systems, and point out practical concerns such as run-time overhead, model drift and limited explainability. Kahu [14] trained an LSTM network on data collected from a running Linux kernel to predict which task the Completely Fair Scheduler will pick next.

2.5 Summary of the surveyed work

Table 1. Summary of the literature survey

Ref.	Authors (Year)	Approach	Limitation with respect to this project
[1]	Silberschatz et al. (2018)	FCFS, SJF, SRTF, RR; exponential averaging; 80% quantum rule	Static algorithms; the 80% rule is applied by hand
[2], [3]	Arpaci-Dusseau (2018); Corbató et al. (1962)	Multilevel feedback queue with periodic priority boost	No burst estimate; quanta per level are still fixed
[4]	Waldspurger & Weihl (1994)	Lottery (proportional-share) scheduling	Targets fairness, not waiting time; ignores burst length
[5]	Matarneh (2009)	RR with quantum = median burst time of current processes	Needs burst times in advance; small static test sets
[6]	Noon et al. (2011)	RR with quantum = mean burst time of waiting processes	Needs burst times in advance; CPU-only processes
[7]	Mohanty et al. (2011)	Per-process “intelligent” time slice from priority and burst time	Needs burst times in advance; aimed at soft real-time systems
[8]	Helmy & Dekdouk (2007)	Burst Round Robin favouring shorter jobs, weight adjustment for I/O	Burst information assumed; no learned prediction
[9]	Negi & Kumar (2005)	C4.5 decision tree learns time-slice use of known programs; modified Linux kernel	Works per known program; small gains (1.4–5.8%); does not reorder the queue
[10]	Pourali & Rahmani (2009)	Fuzzy system predicts next CPU burst for Shortest-Process-Next	Prediction used only for ordering, not for the quantum
[11]	Smith et al. (1998)	Run-time prediction of parallel jobs from history	Whole-job prediction for batch systems, not per-burst
[12]	Mao et al. (2016)	Deep reinforcement learning for cluster resource management	Heavy model; cluster jobs, not CPU bursts
[13]	Zhang et al. (2024)	Survey of AI techniques inside operating systems	Survey; highlights overhead and drift as open issues
[14]	Kahu (2025)	LSTM predicts the next task chosen by Linux CFS	Predicts the scheduler's decision instead of improving it

3. Gaps in Existing Work

Going through the surveyed work, the following gaps were identified.

1. **The quantum is still fixed, or computed from information the scheduler does not have.** Standard Round Robin and MLFQ use quanta chosen in advance. The dynamic quantum

variants [5]–[8] change the quantum, but they compute it from burst times that are assumed to be known, which is the same assumption that makes SJF impractical.

2. **Burst prediction is weak or used only once.** Exponential averaging uses one feature and lags after a phase change. Where better predictors have been tried [9], [10], the prediction is used either for the time slice or for the ordering, but not for both.
3. **Starvation is often not handled.** Any policy that favours short bursts can keep postponing long processes. Several dynamic schemes focus on average waiting time and do not report the worst-case waiting time or the number of starved processes.
4. **Evaluation is limited.** Many papers test on a handful of processes that all arrive at time zero, have a single CPU burst and no I/O, and they ignore context-switch cost. Results are rarely repeated over several random workloads, and there is no reference point showing how close a method gets to the best achievable result.
5. **Learning-based methods are heavy and aimed elsewhere.** Deep learning approaches [12], [14] target clusters or try to imitate an existing scheduler, and the survey in [13] points out their overhead. A light model that predicts individual CPU bursts and is cheap enough to call at every scheduling decision has received little attention.

3.1 How this project addresses the gaps

- The next CPU burst of each process is **predicted** from data the system already has (past bursts, I/O waits, variability), using a small gradient-boosting model, instead of being assumed known.
- The prediction is used **twice**: to order the ready queue and to recompute the time quantum from the 80% rule while the system runs.
- An **ageing term** is added to the ordering so that long processes cannot starve, and the worst-case waiting time is measured explicitly.
- Evaluation uses a **verified simulator** with I/O, context-switch cost, many processes, six workload types and ten random repetitions, with an **oracle** that knows all burst lengths as a reference point.

The proposed methodology and architecture are described in Review 2, and the performance evaluation and comparison in Review 3.

References

[1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ, USA: Wiley, 2018, ch. 5.

[2] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, v1.00. Arpaci-Dusseau Books, 2018, chs. 7–8.

[3] F. J. Corbató, M. Merwin-Daggett, and R. C. Daley, “An experimental time-sharing system,” in *Proc. Spring Joint Computer Conf. (AFIPS)*, 1962, pp. 335–344.

[4] C. A. Waldspurger and W. E. Weihl, “Lottery scheduling: Flexible proportional-share resource management,” in *Proc. 1st USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 1994, pp. 1–11.

[5] R. J. Matarneh, “Self-adjustment time quantum in round robin algorithm depending on burst time of the now running processes,” *American Journal of Applied Sciences*, vol. 6, no. 10, pp. 1831–1837, 2009.

[6] A. Noon, A. Kalakech, and S. Kadry, “A new round robin based scheduling algorithm for operating systems: Dynamic quantum using the mean average,” *International Journal of Computer Science Issues*, vol. 8, no. 3, pp. 224–229, 2011.

[7] R. Mohanty, H. S. Behera, K. Patwari, M. Dash, and M. L. Prasanna, “Priority based dynamic round robin (PBDRR) algorithm with intelligent time slice for soft real time systems,” *International Journal of Advanced Computer Science and Applications*, vol. 2, no. 2, pp. 46–50, 2011.

[8] T. Helmy and A. Dekdouk, “Burst round robin as a proportional-share scheduling algorithm,” in *Proc. IEEE GCC Conference*, 2007.

[9] A. Negi and P. K. Kumar, “Applying machine learning techniques to improve Linux process scheduling,” in *Proc. IEEE Region 10 Conference (TENCON)*, 2005, pp. 1–6.

[10] A. Pourali and A. M. Rahmani, “A fuzzy-based scheduling algorithm for prediction of next CPU-burst time to implement shortest process next,” in *Proc. IACSIT Spring Conference (IACSIT-SC)*, 2009, pp. 217–220.

[11] W. Smith, I. Foster, and V. Taylor, “Predicting application run times using historical information,” in *Job Scheduling Strategies for Parallel Processing*, LNCS vol. 1459. Springer, 1998, pp. 122–142.

[12] H. Mao, M. Alizadeh, I. Menache, and S. Kandula, “Resource management with deep reinforcement learning,” in *Proc. 15th ACM Workshop on Hot Topics in Networks (HotNets)*, 2016, pp. 50–56.

[13] Y. Zhang, X. Zhao, Z. Li, G. Cheng, J. Yin, L. Zhang, and Z. Chen, “Integrating artificial intelligence into operating systems: A survey on techniques, applications, and future directions,” arXiv:2407.14567, 2024.

[14] S. Y. Kahu, “KernelOracle: Predicting the Linux scheduler's next move with deep learning,” arXiv:2505.15213, 2025.